

01-command-line-basics

- Command Line Basics
 - Notes from our first session
 - Why we are using the command line
 - Terminal, shell, and command
 - Where am I?
 - pwd: print the current directory
 - Files and directories
 - ls: list files and directories
 - Paths
 - Absolute path
 - Relative path
 - Reading the information shown by ls -l
 - Changing directories with cd
 - Creating directories with mkdir
 - A working directory such as scratch
 - Copying with cp
 - Moving and renaming with mv
 - Removing files and directories
 - Creating and editing text files
 - nano: create or edit a file
 - touch: create an empty file
 - Reading files
 - cat: print a file
 - less: read a long file page by page
 - head and tail: inspect the beginning or end
 - Searching with grep, pipes, and Ripgrep
 - grep and the pipe -
 - Ripgrep (rg)
 - Bash and .bashrc
 - The I alias from our session
 - Connecting to another Linux machine with ssh
 - Small things that save time
 - Tab completion
 - Command history
 - Stop a running command
 - Clear the terminal
 - Ask a command for help

- [Common messages](#)
 - [No such file or directory](#)
 - [command not found](#)
 - [Permission denied](#)
- [Repeat the exercise from our session](#)
- [Quick reference](#)
- [Before our next session](#)

Command Line Basics

Notes from our first session

Alok, this is a reference for the commands we went through in our first session. There is no need to memorize everything at once. The commands become familiar quite quickly once you start using them while working with actual files.

Linux commands are case-sensitive, so `pwd` and `PWD` are not the same. Commands are normally written in lowercase.

Why we are using the command line

During the session, we compared the graphical user interface (GUI) with the command line interface (CLI). A GUI is useful for many everyday tasks, but simulation work often involves remote Linux machines, many input and output files, and commands that need to be repeated exactly. The CLI gives us more direct control in those situations.

Since your laptop runs Windows, Ubuntu through Windows Subsystem for Linux (WSL) will give you a Linux terminal without replacing Windows. The same basic commands will also work later on a remote workstation or HPC cluster.

Terminal, shell, and command

These words are related, but they do not mean exactly the same thing:

- **Terminal:** the window in which we type commands.
- **Shell:** the program that reads those commands. On Ubuntu, this is commonly Bash.
- **Command:** an instruction such as `pwd`, `cd`, or `mkdir`.

A typical command has this form:

```
command option target
```

For example:

```
ls -lh Documents
```

Here, `ls` is the command, `-lh` contains two options, and `Documents` is the directory we want to examine.

Where am I?

At any moment, the shell is working inside one directory. This is called the **current/present working directory**.

`pwd` : print the current directory

```
pwd
```

We used "present working directory" as an easy way to remember `pwd`. In Linux manuals, the command is usually expanded as **print working directory**. Both point to the same idea: show the directory in which the terminal is currently working.

Example output:

```
/home/alok
```

It is a good habit to use `pwd` before copying, moving, or removing files. It confirms where you are.

Files and directories

A **file** contains information, such as a LAMMPS input, a log, a trajectory, or a text note. A **directory** organizes files and other directories. The words *directory* and *folder* refer to the same thing here.

Linux does not require filename extensions, but clear extensions make files easier to recognize. For example:

- `LJ_argon.in` : LAMMPS input
- `LJ_argon.log` : run log
- `run.sh` : shell script
- `thermo.csv` : tabular thermo data

`ls` : list files and directories

```
ls
```

Some useful forms are:

```
ls -l
ls -lh
ls -a
ls -lah
ls -lrth
alias l="ls -lrth"
```

- `-l` shows details such as permissions, owner, size, and modification time.
- `-h` shows file sizes in a readable form such as KB, MB, or GB.
- `-a` also shows hidden files. Hidden names begin with a dot, for example `.bashrc`.
- `-t` sorts by modification time, with newer items first.
- `-r` reverses the order. In `ls -lrth`, this puts the newest item at the bottom.

The short command `l` that I used during the session is an alias for `ls -lrth`. It is a personal shortcut rather than a standard Linux command. We will return to aliases in the `.bashrc` section.

You can also examine another directory without moving into it:

```
ls -lh /home/alok/Documents
```

Paths

A path tells Linux where a file or directory is located.

Absolute path

An absolute path starts from `/`, the root of the filesystem:

```
/home/alok/simulations/argon
```

It identifies the same location regardless of your current directory.

Relative path

A relative path starts from the current directory:

```
simulations/argon
```

These shortcuts are useful:

Symbol	Meaning
•	current directory
••	parent directory, one level above
~	your home directory
/	root of the Linux filesystem

For example:

```
ls ..  
ls ~/Documents
```

If a path contains spaces, place it inside quotes:

```
cd "My Simulations"
```

This is why I usually put hyphens in file and directory names, for example `argon-simulation` instead of `argon simulation`. It avoids having to use quotes repeatedly.

Reading the information shown by `ls -l`

During the session, we briefly looked at an entry such as:

```
-rw-r--r--  1 alok alok  245 Jun 20 10:30 file1.txt
```

The first character tells us the item type:

- `-` means a regular file;
- `d` means a directory.

The next nine characters describe permissions for the owner, group, and others:

- `r` : read;
- `w` : write or modify;
- `x` : execute;
- `-` : that permission is absent.

We do not need to change permissions yet. For now, it is enough to recognize why `ls -l` shows `r`, `w`, and `x` beside a file.

Changing directories with `cd`

Move into a directory:

```
cd Documents
```

Move one level up:

```
cd ..
```

Return to your home directory:

```
cd ~
```

or simply:

```
cd
```

Return to the previous directory:

```
cd -
```

A useful pattern is to change directory and then confirm the location:

```
cd ~/simulations/argon  
pwd  
ls -lh
```

Creating directories with `mkdir`

Create one directory:

```
mkdir argon
```

Create several directories:

```
mkdir input output analysis
```

Create a complete directory path, including missing parent directories:

```
mkdir -p simulations/argon/analysis
```

The `-p` option is convenient when the intermediate directories may not exist yet.

A working directory such as `scratch`

We used a `scratch` directory during the session. It is a place for active calculations, temporary copies, and experiments. Once a calculation or analysis becomes important, keep a separate copy in backed-up or long-term storage. This makes it easier to experiment without confusing temporary files with final results.

Copying with `cp`

Copy a file:

```
cp input.in input_backup.in
```

Copy a file into another directory:

```
cp input.in backup/
```

Copy a directory and everything inside it:

```
cp -r argon argon_backup
```

The `-r` option means recursive. It is needed when copying a directory.

When you are unsure whether a file with the same name already exists, use interactive mode:

```
cp -i input.in backup/
```

This asks before overwriting an existing file.

Moving and renaming with `mv`

`mv` is used both to move something and to rename it.

Rename a file:

```
mv old_name.in LJ_argon.in
```

Move a file into another directory:

```
mv LJ_argon.in input/
```

Move a directory:

```
mv old_results archive/
```

Use interactive mode if an overwrite is possible:

```
mv -i LJ_argon.in input/
```

Removing files and directories

Removal commands need care. Files removed with `rm` normally do not go to a recycle bin.

Remove a file:

```
rm old_file.txt
```

Ask for confirmation first:

```
rm -i old_file.txt
```

Remove an empty directory:

```
rmdir empty_directory
```

Remove a directory and its contents:

```
rm -r old_results
```

The command `rm -rf` removes recursively without asking for confirmation. It is useful in some situations, but a wrong path can remove a large amount of work. Before using it, check the target with `pwd` and `ls`:

```
pwd
ls -lh old_results
rm -r old_results
```

Be especially careful with `*`, which means "all matching names":

```
rm *.log
```

This removes every `.log` file in the current directory. First check what will match:

```
ls *.log
```

Creating and editing text files

`nano` : create or edit a file

We used Nano because it is a simple terminal text editor. If the file exists, Nano opens it. If it does not exist, Nano creates it when you save.

```
nano file1.txt
```

Inside Nano:

- type normally to add or change text;
- press Ctrl+O to write the changes to the file;
- press Enter to confirm the filename;
- press Ctrl+X to leave Nano.

The shortcuts shown at the bottom use `^` for Ctrl. Therefore, `^O` means Ctrl+O and `^X` means Ctrl+X.

`touch` : create an empty file

```
touch empty-file.txt
```

Unlike Nano, `touch` does not open an editor. It simply creates an empty file if the file does not already exist. If it exists, `touch` updates its modification time.

Reading files

`cat` : print a file

```
cat LJ_argon.in
```

`cat` is convenient for short files. For a long file, the text may move past the screen too quickly.

`less` : read a long file page by page

```
less LJ_argon.log
```

Inside `less` :

- use the arrow keys or Page Up/Page Down to move;
- type `/temperature` to search for `temperature` ;
- press `n` to move to the next match;
- press `q` to quit.

`head` and `tail` : inspect the beginning or end

```
head LJ_argon.in
tail LJ_argon.log
```

Choose the number of lines with `-n` :

```
head -n 20 LJ_argon.in
tail -n 40 LJ_argon.log
```

The end of a simulation log often contains useful information about whether the run completed.

Searching with `grep` , pipes, and `Ripgrep`

`grep` and the pipe `|`

In the session, we first printed a file with `cat` and sent that output to `grep` :

```
cat file1.txt | grep "hello"
```

The pipe `|` sends the output of the command on its left to the command on its right. Here, `cat` prints the file and `grep` keeps only lines containing `hello` .

`grep` can also read the file directly:

```
grep "hello" file1.txt
```

Ripgrep (rg)

`rg` is the command provided by **Ripgrep**. It searches through text files and is very fast, even inside a directory containing many files.

Search for a word in files below the current directory:

```
rg "temperature"
```

We can use options/flags to modify behaviour of `rg`.

To show matching line numbers:

```
rg -n "temperature"
```

To ignore uppercase/lowercase differences:

```
rg -i "error"
```

To search inside a particular directory:

```
rg -n "pair_style" simulations/
```

To search only selected files:

```
rg -n "pair_style" -g "*.in"
```

List files below the current directory:

```
rg --files
```

Often you will find that your simulations are going to fail for whatever reason and you can look for certain keywords, to solve the prob, in the log file using errors or warning keywords. For simulation work, some useful searches are:

```
rg -n "ERROR" LJ_argon.log
rg -n "fix|run|timestep" LJ_argon.in
rg --files -g "*.in"
```

In the second command, `|` means "or" within the search pattern: find `fix`, `run`, or `timestep`.

Bash and `.bashrc`

Bash is the shell that interprets the commands entered in many Ubuntu terminals. It can also read a file containing several commands and run them in order. Such a file is called a shell script and commonly uses the `.sh` extension:

```
bash run.sh
```

The `.bashrc` file is a personal Bash configuration file in your home directory:

```
~/ .bashrc
```

The leading dot makes it hidden. You can see it with:

```
ls -la ~
```

`.bashrc` is commonly used for settings that should be available whenever an interactive Bash terminal starts, such as:

- adding software directories to `PATH`;
- defining short aliases;
- setting environment variables;
- changing the appearance of the command prompt.

For example:

```
export PATH="$HOME/bin:$PATH"
```

This tells Bash to also look in `~/bin` when searching for executable programs.

After changing `.bashrc`, apply the changes to the current terminal with:

```
source ~/.bashrc
```

Otherwise, the changes will normally appear when a new terminal is opened.

It is sensible to keep a backup before editing this file:

```
cp ~/.bashrc ~/.bashrc.backup
```

Lines beginning with `#` are comments. Bash does not execute them; they are there to explain the configuration to a person reading the file.

The `l` alias from our session

The alias we used was:

```
alias l="ls -lrth"
```

After running this command, entering `l` is equivalent to entering `ls -lrth`. An alias entered directly in a terminal lasts only for that shell session. To keep it for future terminals, add the alias to `~/.bashrc` and then run:

```
source ~/.bashrc
```

Connecting to another Linux machine with `ssh`

We briefly connected from the Mac terminal to the Ubuntu workstation. The general form is:

```
ssh username@ip-address
```

For a first connection, we normally need the username on the remote machine, its IP address or hostname, and the user's password. SSH keys can later be configured so that a trusted computer does not need the password for every connection.

After connecting, commands run on the remote machine rather than the laptop in front of you. The command prompt usually changes, so it is worth checking:

```
pwd  
ls -lah
```

Leave the remote machine with:

```
exit
```

Small things that save time

Tab completion

Type the first few letters of a file or directory and press Tab. Bash completes the name when possible. Pressing Tab twice shows available matches.

This is faster and reduces typing mistakes in long paths.

Command history

Use the Up and Down arrow keys to revisit commands. You can also view the history:

```
history
```

Search previous commands interactively with Ctrl+R, type part of the command, and press Enter when the required command appears.

Stop a running command

Press Ctrl+C to interrupt a command that is currently running. This does not close the terminal.

Clear the terminal

```
clear
```

The keyboard shortcut `Ctrl+L` does the same thing in Bash.

Ask a command for help

Many commands provide a short help page:

```
cp --help  
rg --help
```

For a more detailed manual page:

```
man cp
```

Press `q` to leave the manual.

Common messages

```
No such file or directory
```

The path may be wrong, the name may use different uppercase/lowercase letters, or you may be in a different directory. Check with:

```
pwd  
ls -lah
```

```
command not found
```

Bash could not find the command. First check the spelling. If the spelling is correct, the program may not be installed or its location may not be included in `PATH`.

```
Permission denied
```

Your user account may not have permission to read, change, or run the file. Check its permissions with:

```
ls -l filename
```

It is better to understand which permission is missing before trying to force the command.

Repeat the exercise from our session

This follows the same sequence we used together. It can be run line by line in Ubuntu through WSL.

Create a separate practice directory:

```
cd ~  
mkdir cli-practice  
cd cli-practice  
pwd  
ls -lah
```

Create the first file:

```
nano file1.txt
```

Write two or three lines, including the word `hello`. Save with Ctrl+O, confirm with Enter, and leave Nano with Ctrl+X. Then read the file:

```
cat file1.txt
```

Search for the word and create a copy:

```
grep "hello" file1.txt  
rg "hello" file1.txt  
cp file1.txt file2.txt  
ls -lrth
```

Rename the copy and create an empty file:

```
mv file2.txt Alok.txt  
touch empty-file.txt  
ls -lrth
```

Remove the two extra files, then leave and remove the practice directory:

```
rm -i Alok.txt  
rm -i empty-file.txt  
cd ~
```

```
pwd
ls -ld cli-practice
rm -r cli-practice
```

Quick reference

Command	Purpose	Example
pwd	show the current directory	pwd
ls	list files and directories	ls -lah
cd	change directory	cd ~/simulations
mkdir	create a directory	mkdir -p argon/analysis
nano	create or edit a text file	nano file1.txt
touch	create an empty file	touch empty-file.txt
cp	copy a file	cp input.in input.backup.in
cp -r	copy a directory	cp -r argon argon_backup
mv	move or rename	mv old.in LJ_argon.in
rm	remove a file	rm -i old.log
rmdir	remove an empty directory	rmdir empty_directory
rm -r	remove a directory and its contents	rm -r old_results
cat	print a short file	cat LJ_argon.in
less	read a long file	less LJ_argon.log
head	show the first lines	head -n 20 LJ_argon.in
tail	show the last lines	tail -n 40 LJ_argon.log
grep	search for matching lines	grep "hello" file1.txt
rg	search inside files	rg -n "ERROR" .
rg --files	list files recursively	rg --files
alias	define a command shortcut	alias l="ls -lrth"
ssh	connect to a remote machine	ssh username@ip-address
exit	leave a shell or SSH connection	exit
history	show previous commands	history
clear	clear the terminal display	clear
source	read commands into the current shell	source ~/.bashrc

The three checks worth developing into a habit are simple:

```
pwd
ls -lh
```

```
# then run the command
```

They take only a few seconds and prevent many path and filename mistakes.

Before our next session

Repeat the short exercise a few times in Ubuntu/WSL. The aim is not to remember every option. It is to become comfortable checking the current directory, creating and editing a file, making a copy, renaming it, searching its contents, and removing it carefully.

In the next session, we can use the same operations to build a small LAMMPS input for Argon. Argon gives us a clean starting point for seeing how atoms and a Lennard-Jones interaction are described before we move toward bonded systems such as water.